

Introdução à Programação orientada a objetos

Marcos Monteiro Junior

Programando com Java

Sumário

1	Programação em Java	3
1.1	Criando um projeto	3
1.2	Entrada e saída em Java via console	4
1.3	Tipos primitivos de dados em Java	6
1.4	Casting	7
1.4.1	Conversão implícita	7
1.4.2	Conversão explícita	7
1.5	Aritmética: operadores de igualdade e operadores relacionais	8
1.6	Tomada de decisão	9
1.6.1	O comando switch	10
1.7	Estruturas de repetição	11
1.7.1	A estrutura for-each	11
1.8	Arrays	12
1.8.1	Declarando arrays	12
2	Referências Bibliográficas	13

1 Programação em Java

O desenvolvimento em Java exige a configuração de um ambiente próprio, começando pela instalação do JDK (Kit de Desenvolvimento Java, que contém a JVM), disponível no site da Oracle. Para facilitar o processo, é comum utilizar pacotes completos que já unem o JDK a uma IDE (Integrated Development Environment). Ferramentas clássicas como o Eclipse e o NetBeans continuam sendo ótimas escolhas para esse propósito. No entanto, o mercado atual tem migrado fortemente para editores mais leves, como o Visual Studio Code, ou IDEs modernas, como o IntelliJ IDEA (da empresa JetBrains).

A escolha é de acordo com a necessidade e objetivos que se pretendem atingir. Caso o programador opte por VSCode, é possível encontrar o 'kit' de desenvolvimento completo para download. Nessa apostila será utilizado o VScode, assim como o IntelliJ IDE.

1.1 Criando um projeto

Para criação de um projeto utilizando o IntelliJ, basta clicar em **file->new->project**. Escolher a pasta onde vai se localizar o projeto.

Inicialmente iremos utilizar o próprio IntelliJ como gerenciador do projeto. A Figura 1.1 resume essas etapas.

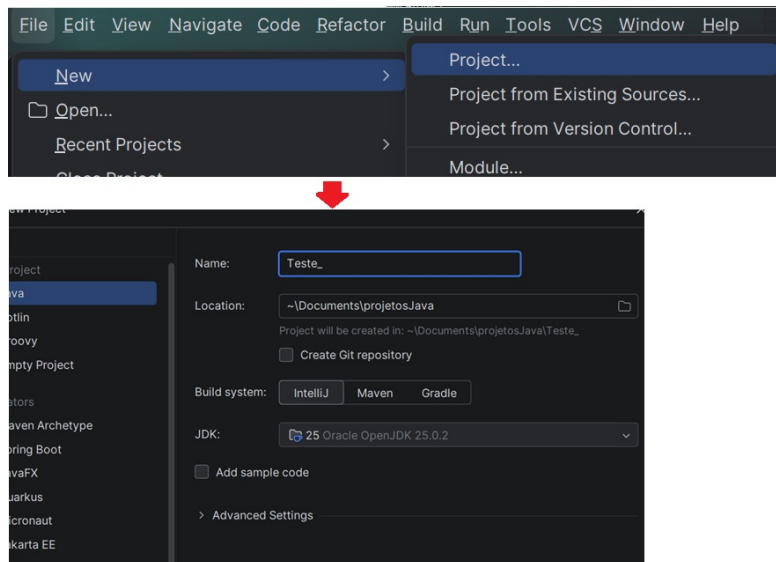


Figura 1.1: Etapas para criação de um Projeto Java.

1 Programação em Java

Depois da criação do projeto, na pasta `src` crie um arquivo do tipo `java.class` de o nome `EntradaSaída`. Na Figura 1.2 está representada essa etapa.

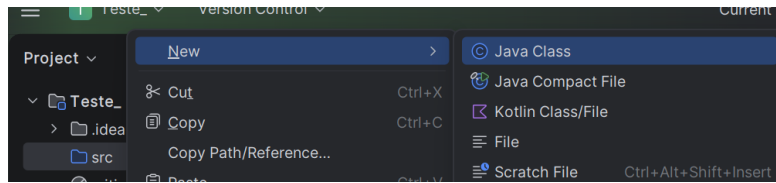


Figura 1.2: Etapas para criação de um arquivo Java class

1.2 Entrada e saída em Java via console

Uma das primeiras etapas no aprendizado de qualquer linguagem de programação é o uso dos recursos de entrada e saída. Em Java, na maioria das versões, é necessário criar uma classe para executar até mesmo o código mais simples. Embora para iniciantes o conceito de classe pareça apenas uma burocracia para garantir a integridade e o formalismo, essa exigência existe porque o Java foi concebido para ser uma linguagem estritamente orientada a objetos. O conceito de classe será discutido nos próximos capítulos.

O arquivo criado na seção anterior deve estar de acordo com a Figura

```
1  import java.util.Scanner;
2
3  public class EntraSaida { no usages
4      |
5  }
```

Figura 1.3: Resultado de um arquivo Java class.

Vamos complementar esse arquivo com alguns códigos. O primeiro código a ser adicionado é: `public static void main(String[] args){ ... }`. Esse trecho de código representa um **método estático público** que diz para a JVM que o projeto deve ser iniciado por esse arquivo.

Abaixo está a explicação de cada uma das palavras reservadas usadas nesse código.

- **public** (Público): É um modificador de acesso. Ele indica que este método é público e pode ser acessado de qualquer lugar.
- **static** (Estático): Indica que este método pertence à classe, e não a um objeto (uma instância da classe).
- **void** (Vazio): É o tipo de retorno.

1 Programação em Java

- `main` (Principal): É o nome do método. Não é uma palavra reservada da linguagem, mas é o nome padronizado e obrigatório que a JVM foi programada para procurar.
- `String[] args` (Argumentos): São os parâmetros que o método recebe.
- `String[]` indica que é um vetor (array) de textos.
- `args` é apenas o nome da variável. Se você executar o programa pelo terminal passando informações extras (ex: `java MeuPrograma arquivo.txt`), esse "`arquivo.txt`" entrará no programa através dessa variável.

Agora vamos adicionar uma saída de texto no console. Para isso, utilizamos o objeto `System.out`. Os três métodos mais comuns são:

- `System.out.print()`: Imprime o texto no console, mas mantém o cursor na mesma linha.
- `System.out.println()`: Imprime o texto e, ao final, quebra a linha (move o cursor para a linha de baixo). É o mais utilizado.
- `System.out.printf()`: Usado para impressões formatadas (similar à linguagem C), onde você usa marcadores como `%s` para textos e `%d` para números inteiros.

Adicione a mensagem clássica de “Olá mundo” no método `System.out`, ficando assim: `System.out.println("Olá mundo");`. O código final deve ficar como na Figura 1.4

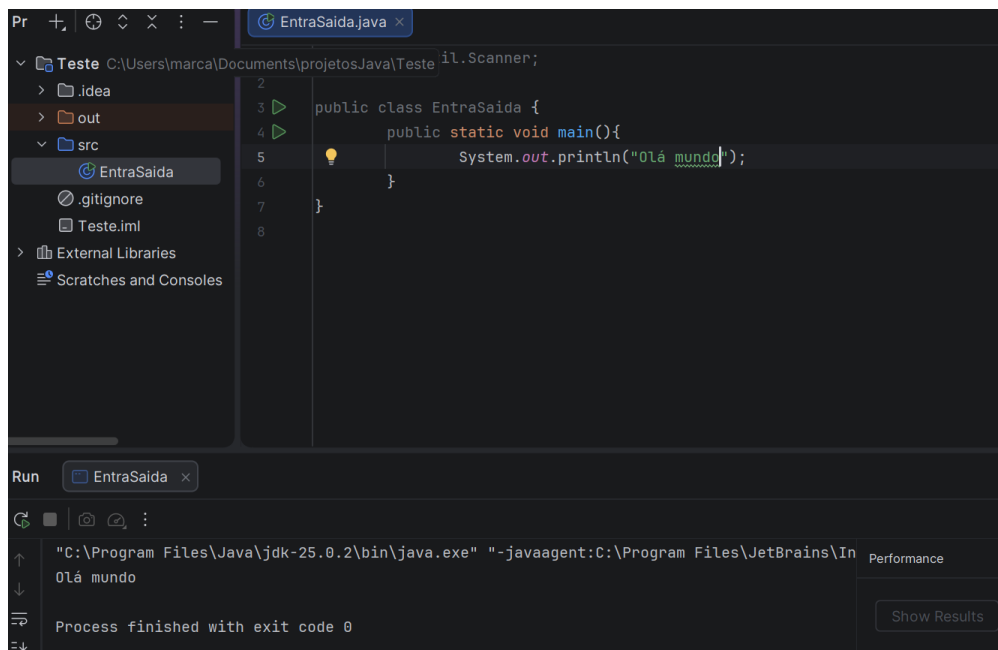


Figura 1.4: Primeira aplicação em Java.

Por outro lado, a entrada de dados em Java via console é feita utilizando a classe `Scanner`. Devemos adicionar um import dessa classe - `import java.util.Scanner;`. Precisamos criar um objeto (discutiremos mais adiante como isso funciona).

A classe `Scanner` tem vários métodos, sendo os mais comuns:

- `nextLine()`: Lê um texto (uma linha inteira) até o usuário apertar Enter.
- `nextInt()`: Lê um número inteiro.
- `nextDouble()`: Lê um número decimal.

Veja o exemplo abaixo:

```
import java.util.Scanner; // 1. Importacao necessaria
public class EntradaSaida {
    public static void main(String[] args) {
        // 2. Instanciando o Scanner
        Scanner teclado = new Scanner(System.in);
        System.out.println("Qual e o seu nome?");
        String nome = teclado.nextLine(); // Le a linha digitada
        // --- SAIDA E ENTRADA DE NUMERO INTEIRO ---
        System.out.println("Quantos anos voce tem?");
        int idade = teclado.nextInt(); // Le o numero digitado
        // --- SAIDA E ENTRADA DE NUMERO DECIMAL ---
        System.out.println("Qual e a sua altura?");
        double altura = teclado.nextDouble(); // Le o decimal
            digitado
        // --- SAIDA FINAL FORMATADA ---
        System.out.println("\n--- Resumo dos Dados ---");
        System.out.println("Nome: " + nome);
        System.out.println("Idade: " + idade + " anos");
        System.out.println("Altura: " + altura + "m");
        // 3. Boa pratica: fechar o Scanner ao final do uso
        teclado.close();
    }
}
```

1.3 Tipos primitivos de dados em Java

Em Java, os tipos primitivos são os “blocos de construção” mais básicos da linguagem. Eles representam valores simples (como números e caracteres) e não são objetos. Por isso, são armazenados diretamente na memória Stack, o que os torna extremamente rápidos e eficientes.

Java possui 8 tipos primitivos, divididos basicamente em quatro categorias:

- Números Inteiros: `int` armazena números inteiros. Esse tipo possui variações do tipo `double`, `short`, `long`, entre outras. Essas variações determinam o tamanho de espaço de memória que será utilizado.

- Ponto flutuante: -float armazena números com valores flutuantes.
- Caractere: -char armazena um caractere.
- Lógica: -boolean é uma variável de valor verdadeiro ou falso.

A declaração dessas variáveis em Java é feita de forma similar a declaração de variáveis em C. Alguns exemplos abaixo:

- `int numero;`
- `float numero_float;`
- `boolean variavel;`

1.4 Casting

O casting em Java é o processo de converter um valor de um tipo de dado para outro. Como o Java é uma linguagem fortemente tipada, ele não permite que você atribua valores de tipos incompatíveis diretamente, a menos que você realize uma conversão explícita ou que a conversão seja segura (“automática”).

Podemos dividir o casting em dois tipos principais: Conversão Implícita (*Widening*) e Conversão Explícita (*Narrowing*)

1.4.1 Conversão implícita

Ocorre quando você converte um tipo de “menor capacidade” para um de “maior capacidade”. Como não há risco de perder dados (o destino comporta o valor original com folga), o Java faz isso automaticamente.

byte → *short* → *int* → *long* → *float* → *double* O caminho inverso não é permitido.

1.4.2 Conversão explícita

Ocorre quando você quer converter de um tipo de “maior capacidade” para um de “menor capacidade”. Aqui há risco de perda de dados (truncamento), por isso o Java exige que você especifique o casting manualmente, usando parênteses antes do valor.

```
double meuDouble = 9.78;
int meuInt = (int) meuDouble; // Explícito: perderemos o .78
```

Atenção Estouro de Memória (*Overflow*): Se você tentar converter um número muito grande (`int` com valor 500) para um tipo pequeno (`byte`, que só vai até 127), o resultado será um valor “estranho” devido ao estouro de bits. É um excelente exemplo prático para mostrar aos alunos por que o casting explícito é perigoso.